

② Fidé : Logiciel de gestion des résultats dans les tournois d'échecs

On s'intéresse ici au logiciel Fidé qui permet de gérer les résultats des différents tournois d'échecs qui sont organisés sous l'égide de la Fédération Internationale Des Echecs. Ce logiciel repose sur une base de données Oracle dont le schéma relationnel vous est communiqué ci-dessous.

LIGUES (idLigue, nomLigue, zoneLigue, *nbJoueursLigue*)

CLUBS (idClub, nomClub, idLigue#)

JOUEURS (idJoueur, nomJoueur, prenomJoueur, sexeJoueur, dateNaissanceJoueur, eloJoueur, idClub#)

TOURNOIS (idTournoi, nomTournoi, lieuTournoi, nbRondesTournoi, *nbParticipantsTournoi*)

PARTICIPER (idTournoi#, idJoueur#)

PARTIES (idPartie, idTournoi#, numRonde, numTable, idJoueurBlancs#, idJoueurNoirs#, resultatPartie)

La table *Clubs* recense les clubs dans lesquels sont licenciés les joueurs d'échecs. Chaque club est affilié à une ligue régionale (par exemple la ligue du Languedoc, la ligue d'Aquitaine, la ligue d'Ile de France, etc.). Dans la table *Ligues*, on peut trouver toutes les informations qui concernent ces ligues.

La table *Joueurs* contient les informations nominatives qui concernent les différents joueurs licenciés à la Fédération Internationale Des Echecs. Dans cette table, l'attribut *eloJoueur* permet de connaître le classement Elo du joueur. Ce classement, qui est un nombre compris entre 1 000 et 3 000, renseigne sur le niveau du joueur. Plus ce nombre est élevé, plus le joueur est fort – 1 600 : Pousseur de bois ; 1 800 : Mazette ; 2 000 : Bon joueur de club ; 2 200 : Patate Internationale ; 2 300 : Maitre de la Fédération Internationale ; 2 400 : Maitre International ; 2 500 : Grand Maitre International. Chaque joueur est licencié à un et un seul club (et donc rattaché à une seule ligue régionale).

Dans la table *Tournois* on trouve tous les tournois qui sont organisés par la Fédération Internationale des Echecs. Dans cette table, l'attribut *nbRondesTournoi* indique le nombre de rondes (c'est-à-dire de tours, ou de parties – round en anglais) que vont jouer chacun des participants pendant le tournoi. La table *Participer* permet de connaître les joueurs qui ont participé aux différents tournois.

Enfin, la table *Parties* stocke les résultats de toutes les parties qui ont été jouées par les joueurs dans les tournois. Dans cette table l'attribut *idTournoi* indique dans quel tournoi a été jouée la partie. *numRonde* permet de connaître la ronde (ou le tour) à laquelle cette partie a été jouée dans le tournoi ; par exemple, si *numRonde* = 1, cela veut dire que cette partie correspond à la première partie jouée par les joueurs dans le tournoi. Si *numRonde* = 2, il s'agit de la deuxième partie jouée par les joueurs dans le tournoi, etc.

L'attribut *idJoueurBlancs* indique l'identifiant du joueur qui avait les blancs dans la partie. Et *idJoueurNoirs* celui qui avait les noirs. *resultatPartie* donne le résultat de la partie. Il a pour valeur "1-0" si c'est les blancs qui ont gagné (le joueur avec les blancs marque alors 1 point et le joueur avec les noirs 0). "1/2" si la partie est déclarée nulle (les deux joueurs marquent un demi-point). Et "0-1" si c'est les noirs qui ont gagné (le joueur avec les blancs marque 0 point et le joueur avec les noirs 1).

Vous trouverez sur l'ENT le fichier « FIDE.sql » qui vous permettra de générer les tables de la base de données sur votre schéma.

Dans les tables *Ligues* et *Tournois* figurent respectivement les attributs *nbJoueursLigue* et *nbParticipantsTournoi* qui sont indiqués en italique dans le schéma relationnel. Ces attributs sont dits dérivés (au sens UML), c'est-à-dire qu'ils sont calculables à partir de requêtes SQL. Ils engendrent donc de la redondance dans la base de données mais ont tout de même été maintenus car leur contenu est relativement stable. Toutefois, dans le script « FIDE.sql » leur valeur n'a pas encore été initialisée, et sur toutes les lignes, ces attributs ont pour le moment la valeur NULL.

Première partie – Les Blocs PL/SQL :

1) Bloc PL/SQL simple et paquetage DBMS_OUTPUT.

- Ecrire un bloc PL/SQL qui, grâce au paquetage DBMS_OUTPUT, affiche le nombre de joueurs qui appartiennent au club qui a l'identifiant 'C1'.

On rappelle que pour que le paquetage DBMS_OUTPUT fonctionne dans l'interface iSQL*Plus, il faut l'activer avec la commande SET SERVEROUTPUT ON avant la première instruction du bloc PL/SQL (cette option sera valable durant toute la session iSQL*Plus).

- Résultat attendu :

Il y a 6 joueur(s) dans le club C1

2) Utilisation de variables.

- On souhaite maintenant tester le bloc PL/SQL de la question précédente avec d'autres clubs (par exemple les clubs C2, C3, ..., C9, ...). Mais on veut modifier un minimum de lignes de code lorsque on change le club, et ce grâce à l'utilisation d'une variable.

Si cela n'a pas été fait lors de la question précédente, refactorer le code du bloc PL/SQL afin qu'il ne soit utile de modifier qu'une seule ligne de code lorsque on teste un autre club.

- Résultats attendus :

Il y a 6 joueur(s) dans le club C1

Il y a 4 joueur(s) dans le club C2

Il y a 0 joueur(s) dans le club C9

3) Les Exceptions : l'exception NO_DATA_FOUND.

- Dans la question précédente il y a un léger problème. En effet, si l'identifiant du club n'existe pas (par exemple le C11), il est indiqué qu'il y a 0 joueurs dans le club en question. Et on aurait eu la même réponse si on avait indiqué l'identifiant d'un club qui existe bien mais qui n'a encore aucun joueur licencié (par exemple le club C9).

En s'inspirant du bloc PL/SQL écrit lors de la question précédente, écrire un nouveau bloc qui indiquerait le cas échéant qu'aucun club ne possède l'identifiant qui a été indiqué.

On utilisera ici l'exception prédéfinie NO_DATA_FOUND qui est levée automatiquement lorsqu'une requête SELECT ... INTO ne retourne rien (il est à noter que si la requête retourne plusieurs lignes, c'est alors l'exception TOO_MANY_ROWS qui est levée).

- Résultats attendus :

Il y a 1 joueur(s) dans le club C10

Il y a 0 joueur(s) dans le club C9

Il n'y a pas de club C11

4) Structures de contrôle : la conditionnelle (IF ... THEN ... ELSE ... END IF).

- Ecrire un nouveau bloc PL/SQL qui fait exactement la même chose que la question précédente mais en utilisant cette fois-ci la structure de contrôle (IF ... THEN ... ELSE ... END IF) à la place de la section EXCEPTION.

- Résultats attendus :

Il y a 1 joueur(s) dans le club C10

Il y a 0 joueur(s) dans le club C9

Il n'y a pas de club C11

5) Les variables %ROWTYPE.

- Ecrire un bloc PL/SQL qui permet d'afficher les informations de la table *Tournois* qui concernent le tournoi T1.

Pour se simplifier la tâche, et pour faciliter les éventuelles futures maintenances de l'application, on récupèrera les données qui concernent le tournoi T1 dans une variable %ROWTYPE.

- Résultat attendu :
Identifiant du tournoi : T1
Nom du tournoi : Tournoi de Noël
Lieu du tournoi : Montpellier
Nombre de rondes du tournoi : 5

Deuxième partie – Les Procédures et Fonctions stockées :

Fonctions stockées

6) Fonction stockée nbJoueursParClub.

- En vous inspirant de ce que vous avez écrit lors des questions 3 ou 4, écrire une fonction stockée nbJoueursParClub qui retourne le nombre de joueurs qui appartiennent au club dont l'identifiant p_idClub est passé en paramètre. Cette fonction doit avoir la signature suivante :
FUNCTION nbJoueursParClub(p_idClub IN Clubs.idClub%TYPE) **RETURN** NUMBER
 Si on passe en paramètre de cette fonction un identifiant de club qui n'existe pas, on souhaite que la fonction retourne NULL (et non pas 0).

Si lors de la création de votre fonction, vous avez une erreur de compilation (Warning: Function created with compilation errors) vous pouvez demander à Oracle de vous afficher la liste des erreurs grâce à l'instruction SHOW ERRORS.

On rappelle que sous Oracle, pour tester une fonction stockée, on peut utiliser la pseudo-table DUAL.

- Résultats attendus :

```
SELECT nbJoueursParClub('C1')
FROM DUAL ;

nbJoueursParClub('C1')
-----
6

SELECT nbJoueursParClub('C9')
FROM DUAL ;

nbJoueursParClub('C9')
-----
0

SELECT nbJoueursParClub('C11')
FROM DUAL ;

nbJoueursParClub('C11')
-----

// quand le résultat est NULL, par défaut, iSQL*Plus n'affiche rien
```

7) Fonction stockée nbJoueursParLigue.

- Ecrire une fonction stockée nbJoueursParLigue qui retourne le nombre de joueurs qui sont rattachés (ou dont le club est rattaché) à une ligue dont l'identifiant p_idLigue est passé en paramètre. Cette fonction doit avoir la signature suivante :
FUNCTION nbJoueursParLigue(p_idLigue IN Ligues.idLigue%TYPE)
RETURN NUMBER

Dans le corps de cette fonction, on utilisera la fonction qui a été réalisée à la question précédente (nbJoueursParClub).

On ne vous demande pas de gérer explicitement le cas où l'identifiant de la ligue qui est passé en paramètre n'existe pas, ou bien le cas où la ligue existe mais qu'elle n'a pas de club (dans ces cas-là, si vous utilisez dans votre code la fonction SUM, votre fonction devrait retourner NULL par défaut sans que vous soyez obligé de le programmer).

- Résultat attendu :

```
SELECT nbJoueursParLigue('LAN')
FROM DUAL ;

nbJoueursParLigue('LAN')
-----
14
```

- Ecrire une requête SQL qui, en utilisant la fonction écrite ci-dessus, met à jour l'attribut nbJoueursLigue de toutes les lignes de la table *Ligues*.

Puis, vérifier que dans la table *Ligues* on a bien 14 joueurs dans la ligue du Languedoc.

```
SELECT nbJoueursLigue
FROM Ligues
WHERE nomLigue = 'Languedoc' ;

nbJoueursLigue
-----
14
```

Procédures stockées

8) Procédure stockée affichageInfosTournoi.

- En vous inspirant de ce que vous avez écrit lors de la question 5, écrire une procédure stockée affichageInfosTournoi qui affiche les informations qui concernent le tournoi dont l'identifiant p_idTournoi est passé en paramètre. Cette procédure doit avoir la signature suivante :

```
PROCEDURE affichageInfosTournoi(
                                p_idTournoi IN Tournois.idTournoi%TYPE)
```

On rappelle que sous Oracle, on peut appeler une procédure stockée avec l'instruction CALL.

- Résultat attendu :

```
CALL affichageInfosTournoi('T1');
Identifiant du tournoi : T1
Nom du tournoi : Tournoi de Noël
Lieu du tournoi : Montpellier
Nombre de rondes du tournoi : 5

CALL affichageInfosTournoi('T4');
Le tournoi T4 n'existe pas
```

9) Procédure stockée miseAJourNbParticipantsTournoi.

- Ecrire une procédure stockée qui met à jour l'attribut nbParticipantsTournoi de toutes les lignes de la table *Tournois*. Pour connaître les participants à un tournoi, on pourra interroger la table *Participer*. Votre procédure devra donc avoir une requête UPDATE qui pourra contenir une requête imbriquée corrélée. Cette procédure doit avoir la signature suivante :

```
PROCEDURE miseAJourNbParticipantsTournoi
```

- Résultat attendu :

```
CALL miseAJourNbParticipantsTournoi() ;

SELECT idTournoi, nomTournoi, nbParticipantsTournoi
FROM Tournois;

idTournoi  nomTournoi          nbParticipantsTournoi
-----
T1          Tournoi de Noël       10
T2          Tournoi des Normes   16
T3          Open JCL        14
```